GmSSL 的 SM2 加解密与签名验签课堂测试

姓名：蔡贸俊　学号：20241328　班级：2413

实验日期：2026 年 10 月 26 日

一、实验目的与完成情况

本次实验使用同一份学号文件，练习 SM2 公钥加密、私钥解密、私钥签名和公钥验签。重点是区分加密与签名的作用，并通过文件比对和篡改对照检查程序结果。

题目要求	实现及证据	结果
Ubuntu 或 openEuler	openEuler 24.03 LTS，WSL2	已完成
全班八位学号，本人放最后	sn.txt，29 个不同学号，最后为 20241328	已完成
GmSSL 命令生成密钥对（4 分）	gmssl sm2keygen，产生 PEM 公钥和加密私钥	已完成
GmSSL 编程加解密（10 分）	C 程序调用 sm2_encrypt / sm2_decrypt，diff 返回 0	已通过
GmSSL 编程签名验签（10 分）	C 程序对完整文件签名，原文验签通过、篡改后拒绝	已通过
实践、AI 记录及 Markdown/PDF	本报告与 logs 中的文本记录	已整理

二、实验环境与准备

实际系统为 **openEuler 24.03 LTS（WSL2，x86_64）**，编译器为 **GCC 12.3.1**，密码库为 **GmSSL 3.3.0-dev.1183**。C 程序编译为 Linux ELF 64 位文件。

本次参考以前实验中的 SM2 密钥读取、加密和签名接口，并继续使用 `/root/hash_hw/sn.txt` 中的班级学号数据。每行一个八位学号，共 29 行，采用 LF 换行，最后一行也有换行。文件大小为 29 × 9 = **261 字节**，本人学号放到最后。采集脚本检查了数量、格式、唯一性与最后一行。

```
20241301
20241302
20241303
20241304
20241305
20241306
20241307
20241308
20241309
20241310
20241311
20241312
20241313
20241314
20241315
20241316
20241317
20241318
20241319
20241320
20241321
20241322
20241323
20241324
20241325
20241326
20241327
20241329
20241328
```

三、实验原理与方案

1. 加密和签名的区别

- **加密**：用公钥将原文变为密文，只有持有相应私钥才能解密，主要解决内容保密问题。
- **签名**：用私钥对消息生成签名，用公钥检查签名，主要用于验证消息来源与内容是否发生变化。签名本身不隐藏原文。

实验使用同一对 SM2 密钥演示这两组操作。程序从命令产生的 PEM 文件读取密钥，密钥生成步骤没有用库函数代替题目要求的命令。

2. 261 字节文件的加密处理

本机 `gmssl/sm2.h` 定义 `SM2_MAX_PLAINTEXT_SIZE` 为 255，`gmssl sm2encrypt -help` 也说明输入最多 255 字节。261 字节的学号文件不能直接当作一个 SM2 明文块处理，因此按 **255 + 6 字节**分成两块，每块独立调用 `sm2_encrypt`。

`sn.enc` 使用一个简单的课堂实验封装格式：

```
4 字节文件标识 SM2F
2 字节大端长度 L1 | 第 1 块 DER 密文
2 字节大端长度 L2 | 第 2 块 DER 密文
```

解密先检查标识和每块长度，再逐块调用 `sm2_decrypt`，按原顺序写入 `sn2.txt`。分块只改变加密处理方式，没有删除或压缩任何学号。该封装是程序自定义格式，并不是 GmSSL 标准的单个 DER 密文，也未提供整份文件的块顺序、块数量认证；这里只用于本次文件往返实验。

3. 对完整文件签名

签名采用 `sm2_sign_init/update/finish`，验签采用 `sm2_verify_init/update/finish`。文件数据分次送入 `update`，但最终只生成**一份覆盖整个 261 字节文件的签名**，不受 SM2 单次加密的 255 字节限制。

SM2 签名使用 SM3，并包含与用户标识、公钥和曲线参数有关的 ZA 值。签名与验签统一使用默认标识 `1234567812345678`；程序直接使用 GmSSL 的签名上下文处理 ZA 与摘要。

四、使用 GmSSL 命令生成公私钥对（4 分）

私钥口令由环境变量 `SM2_PASS` 提供，日志仅记录变量名。本次调用的命令及实际结果如下：

```
$ gmssl sm2keygen -pass "$SM2_PASS" -out private/sm2_private.pem -pubout sm2_public.pem
[exit code: 0; expected: 0]

$ head -n 1 private/sm2_private.pem
-----BEGIN ENCRYPTED PRIVATE KEY-----
[exit code: 0; expected: 0]
```

`-out` 指定口令加密的 PKCS#8 私钥文件，`-pubout` 指定公钥文件。产生的公钥如下：

```
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoEcz1UBgi0DQgAEbKH7N6aM1byWIgUZpROPi2I+Qj09
zUxddjRoq4HNESRwp6b7ETMBCyl8psegGl4OTf7YV4UsqTTswAImI+bLPA==
-----END PUBLIC KEY-----
```

本地私钥保存在 `private/sm2_private.pem`；原始运行口令仅保存在 openEuler 的本地状态目录，不进入报告或仓库。公开交付包含公钥和生成方法，复现时运行 `bash run.sh` 并输入新的实验口令，即可重新生成密钥并执行实验。

五、GmSSL 编程加密与解密（10 分）

1. 源码与编译

完整源码为 [src/sm2_file.c](), 在线代码地址：

https://github.com/1515720698/cryptosystem/blob/master/20241328-%E8%94%A1%E8%B4%B8%E4%BF%8A-GmSSL%E7%9A%84SM2%E5%8A%A0%E8%A7%A3%E5%AF%86%E4%B8%8E%E7%AD%BE%E5%90%8D%E9%AA%8C%E7%AD%BE/src/sm2_file

同一个程序提供 `encrypt`、`decrypt`、`sign`、`verify` 四种操作。公钥通过 `sm2_public_key_info_from_pem` 读取，私钥通过 `sm2_private_key_info_decrypt_from_pem` 读取。

```
gcc -std=c11 -Wall -Wextra -O2 -I/usr/local/include \
    src/sm2_file.c -L/usr/local/lib -Wl,-rpath,/usr/local/lib \
    -lgmssl -o sm2_file
```

实际编译没有警告，退出码为 0。`-I` 指定头文件目录，`-L` 指定库目录，`-lgmssl` 链接 GmSSL，`rpath` 指定运行时库搜索路径。

2. 关键处理

下面是程序对每块数据的主要调用，`plain` 数组的长度为 `SM2_MAX_PLAINTEXT_SIZE`：

```
len = 0;
if (sm2_encrypt(key, plain, n, cipher, &len) != 1) goto end;
header[0] = (uint8_t)(len >> 8);
header[1] = (uint8_t)len;
```

解密按长度读出一块 DER 密文，再恢复原始字节：

```
n = 0;
if (sm2_decrypt(key, cipher, len, plain, &n) != 1) goto end;
if (n > sizeof(plain) || fwrite(plain, 1, n, out) != n) goto end;
```

3. 实际运行结果

```
$ ./sm2_file encrypt sm2_public.pem sn.txt sn.enc
encrypt block 1: plaintext=255, DER=365 bytes
encrypt block 2: plaintext=6, DER=112 bytes
encrypt: 2 blocks, plaintext total=261 bytes -> sn.enc
[exit code: 0; expected: 0]

$ ./sm2_file decrypt private/sm2_private.pem sn.enc sn2.txt
decrypt block 1: DER=365, plaintext=255 bytes
decrypt block 2: DER=112, plaintext=6 bytes
decrypt: 2 blocks, plaintext total=261 bytes -> sn2.txt
[exit code: 0; expected: 0]

$ diff sn.txt sn2.txt
[exit code: 0; expected: 0]
```

本次两个 DER 密文分别为 365 和 112 字节，加上 4 字节标识及两个 2 字节长度字段，总密文文件大小为 **485 字节**。SM2 加密带有随机性，DER 坐标编码长度也可能变化，因此重新运行时密文及其长度可以不同。

```
$ sha256sum sn.txt sn2.txt
3e6ae0ce6045401bc073e36c4442f0de3d44cb6f6414ef0d339a5bac386ac012  sn.txt
3e6ae0ce6045401bc073e36c4442f0de3d44cb6f6414ef0d339a5bac386ac012  sn2.txt
[exit code: 0; expected: 0]
```

`diff` 无差异且返回 0，两个 SHA-256 相同，说明包括学号顺序及换行在内的原始文件内容已完整恢复。

六、GmSSL 编程签名与验签（10 分）

1. 关键接口

签名初始化时传入私钥及默认用户标识，读取文件后持续调用 `sm2_sign_update`，最后调用 `sm2_sign_finish` 输出 DER 签名。

```
sm2_sign_init(&ctx, key, SM2_DEFAULT_ID, SM2_DEFAULT_ID_LENGTH);
/* Read the complete file and feed every chunk to sm2_sign_update. */
sm2_sign_update(&ctx, buf, n);
sm2_sign_finish(&ctx, sig, &siglen);
```

验签以公钥初始化，读取同一文件，再用 `sm2_verify_finish` 检查签名。完整源码检查了上述接口的返回值，区分验签成功、拒绝及文件读取错误。

2. 实际运行

```
$ ./sm2_file sign private/sm2_private.pem sn.txt sn.sig
sign: 261 bytes, signature=72 bytes -> sn.sig
[exit code: 0; expected: 0]

$ ./sm2_file verify sm2_public.pem sn.txt sn.sig
verify: PASS, file=sn.txt, bytes=261, id=1234567812345678
[exit code: 0; expected: 0]
```

本次签名为 **72 字节**，保存为 `sn.sig`。签名长度不等于文件长度；本机头文件给出的签名缓冲上限为 72 字节。本次原文验签返回 `PASS`，进程退出码为 0。

3. 篡改对照

将原文第 8 个字节从数字 `1` 改为 `9`，保存为 `logs/sn_tampered.txt`，保持原签名不变，然后执行：

```
$ ./sm2_file verify sm2_public.pem logs/sn_tampered.txt sn.sig
/root/GmSSL-src/src/sm2_sign.c:265:sm2_fast_verify():
/root/GmSSL-src/src/sm2_sign.c:671:sm2_verify_finish():
verify: REJECT, file=logs/sn_tampered.txt, bytes=261, id=1234567812345678
[exit code: 2; expected: 2]
```

程序返回 `REJECT`，退出码为 2，符合预期。日志中的 GmSSL 源文件位置是库在拒绝不匹配签名时输出的诊断，不能把这个预期拒绝误记成原文验签失败。该对照说明本次签名能够识别消息内容的变化。

七、交叉验证与问题分析

1. GmSSL 命令行互验

从自定义封装中提取出两块标准 DER 密文，分别使用 `gmssl sm2decrypt` 解密并拼接，随后与原文进行 `diff`，结果为 0。此外，用命令行验证 C 程序签名，再用 C 程序验证命令行签名：

```
$ gmssl sm2verify -pubkey sm2_public.pem -id 1234567812345678 -in sn.txt -sig sn.sig
verify : success
[exit code: 0; expected: 0]

$ gmssl sm2sign -key private/sm2_private.pem -pass "$SM2_PASS" -id 1234567812345678 -in sn.txt -out logs/cli.sig
[exit code: 0; expected: 0]

$ ./sm2_file verify sm2_public.pem sn.txt logs/cli.sig
verify: PASS, file=sn.txt, bytes=261, id=1234567812345678
[exit code: 0; expected: 0]
```

两种方向均通过，说明本次公钥、用户标识和签名 DER 格式相互兼容。两者仍使用同一套 GmSSL 实现，属于接口与格式交叉验证。

2. 密文截断检查

额外删除密文文件末尾的 1 字节，解密程序检测到第二块长度不足，返回 1，并删除不完整的输出文件。这项检查用于验证程序对不完整输入的处理；它不代表自定义文件格式已经具有整文件完整性保护。

3. 本次需要注意的条件

1. **不能直接加密 261 字节**：先核对接口限制，采用 255 + 6 字节分块，保留全部原文。
2. **签名不等于私钥加密**：分别使用签名接口和加密接口，签名覆盖完整文件。
3. **标识必须保持一致**：C 程序和 CLI 都采用 `1234567812345678`。
4. **不能只看程序是否运行结束**：同时检查返回值、`diff`、正常验签和篡改拒绝。

八、AI 使用与问答记录

本次使用 OpenCode（模型 `openai/gpt-6.1-sol`）辅助实验。提供课堂题目后，提出按前一次作业的方式完成。

- **方法查阅**：AI 检查旧环境和旧代码，读取本机 SM2 头文件与命令帮助，确认 255 字节限制。
- **代码与运行**：AI 协助编写程序，通过工具实际执行密钥生成、编译、文件加解密和签名验签，并保存真实输出。

- **结果核对**：检查文件往返、篡改拒绝、截断拒绝和 CLI 双向签名验证。
- **材料整理**：根据源码与日志编写本报告、AI 使用记录和 PDF，并准备 GitHub 归档。

真实交流节录及过程摘要见 logs/AI使用记录.md，完整运行文本见 logs/practice.txt。本节是实际过程的整理，没有将补写的示例问题当作真实用户提问。

九、实验总结与附件

本次已完成命令生成 SM2 密钥对、程序加解密与签名验签。完整学号文件解密后与原文一致，原文签名验证成功，修改原文后验证被拒绝。

从结果可以归纳：公钥加密与私钥签名使用密钥的方向不同，目的也不同；分块读取可以用于处理文件，但加密接口的单块上限与签名接口处理消息的方式需要分别理解；验证应包含正常结果与不符合预期输入的对照。

文件	用途
src/sm2_file.c	四种核心操作的完整 C 源码
run.sh	在已安装 GmSSL 的 Linux 中生成新密钥、编译、运行
sn.txt / sn2.txt	261 字节原文及解密结果
sn.enc / sn.sig	分块密文封装及完整文件签名
sm2_public.pem	本次运行公钥，可直接复核已有签名
logs/practice.txt	环境、命令、编译及正常运行记录
logs/negative-check.txt	篡改与截断检查
logs/cross-check.txt	CLI 解密和签名双向验证

复现时进入实验目录，执行 `bash run.sh`，按提示输入新私钥口令。程序会生成新密钥及新结果，旧签名应配合旧公钥验证，不能混用不同次运行的密钥与产物。

本次作业 GitHub 目录：https://github.com/1515720698/cryptosystem/tree/master/20241328-%E8%94%A1%E8%B4%B8%E4%BF%8A-GmSSL%E7%9A%84SM2%E5%8A%A0%E8%A7%A3%E5%AF%86%E4%B8%8E%E7%AD%BE%E5%90%8D%E9%AA%8C%E7%AD%BE

参考依据：本次课堂题目；以前的 GmSSL 库编程示例；本机 `gmssl/sm2.h`；实际读取的 `gmssl sm2keygen/sm2sign/sm2verify/sm2encrypt/sm2decrypt -help`。

AI 生成说明：本报告正文及 AI 使用记录由 AI 根据本次实际交流、源码和运行日志生成；代码编写、工具执行、验证和文档整理均包含 AI 辅助。